Extended reading — Week 9: Proximal Policy Optimization

May 21, 2026

1. Schulman, Wolski, Dhariwal, Radford, Klimov (2017) — Proximal Policy Optimization Algorithms

arXiv:1707.06347 (July 2017). 50,000+ citations as of 2026; the most-cited RL paper in history.

Summary. Introduces the PPO-clip and PPO-KL variants. Argues that a first-order approximation to TRPO's trust region, combined with multiple epochs of mini-batch SGD, achieves comparable performance with 5×–10× less wall-clock. The paper has 6 pages of derivation and 4 of empirical results — it is the shortest landmark paper in modern RL.

Why it matters for this week. This is the algorithm we implement. Read Section 3 carefully; pay particular attention to the asymmetry of the clip with respect to the sign of $\hat{A}$.

Annotated key passage.

> *"Note that, despite being a lower bound on L^{CPI} (and hence on the unclipped surrogate $\hat{g}$), L^{CLIP} is not a lower bound when $r_t < 1, \hat{A}_t > 0$, or $r_t > 1, \hat{A}_t < 0$ — in these cases the clip is inactive."*

This subtle remark is what makes the clip a soft trust region rather than a hard one. When the importance ratio moves toward 1 from outside the box, the surrogate gradient is non-zero; only when it moves *further away* does it saturate. The result is that PPO can "re-enter" the trust region even from a violation.

2. Huang, Dossa, Raffin, Kanervisto, Wang (2022) — The 37 Implementation Details of Proximal Policy Optimization

ICLR Blog Track 2022. `iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/`

Summary. A line-by-line forensic comparison of the original Schulman PPO code, the OpenAI Baselines reimplementation, the Stable Baselines3 reimplementation, and CleanRL. Identifies 37 distinct code-level details that affect performance — the orthogonal init, the Adam ε, the value clipping, etc.

Why it matters for this week. The checklist this week's mini-project follows is taken directly from this blog post. Without these details, the canonical PPO loss in Schulman 2017 does not converge.

Annotated key passage.

> *"In 2022, the difference between a paper PPO and a working PPO is 37 lines of code spread across 6 files. Each one is essential."*

A useful corrective to "it's just the loss". The lesson generalises to every RL algorithm: there is no such thing as "the algorithm" in isolation; the code-level details are part of the algorithm.

3. Engstrom, Ilyas, Santurkar, Tsipras, Janoos, Rudolph, Madry (2020) — Implementation Matters in Deep Policy Gradients

ICLR 2020. arXiv:2005.12729.

Summary. See week-8 extended reading. The headline result is reproduced here because it is the empirical justification for the 37-details exercise.

Why it matters for this week. The strongest evidence in the literature that "algorithm" and "implementation" are fundamentally entangled in deep RL.

Annotated key passage.

> *"With the code-level optimisations from PPO added to TRPO, the two algorithms perform indistinguishably on all five MuJoCo benchmarks."*

A controlled experiment: when you hold the optimisations fixed and vary only the algorithm, there is no measurable advantage to either. The corollary for practitioners: do not waste time tuning between PPO and TRPO; tune the 37 details first.

4. Andrychowicz, Raichuk, Stanczyk, Orsini, Girgin, Marinier, Hussenot, Geist, Pietquin, Michalski, Gelly, Bachem (2021) — What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study

ICLR 2021. arXiv:2006.05990.

Summary. A sweep of 250,000 PPO/A2C/IMPALA runs on a battery of continuous-control tasks, ablating individual implementation choices. Findings: orthogonal init matters most for stability; advantage normalisation matters most for final performance; entropy bonus matters most for exploration; the choice of optimiser ($\varepsilon = 10^{-5}$ Adam vs. 10^{-8}) genuinely affects convergence speed.

Why it matters for this week. Quantifies the per-detail contribution. Useful when triaging a broken implementation: read this paper's Figure 4 and decide which detail to fix first.

Annotated key passage.

> *"The default $\varepsilon = 10^{-8}$ in Adam causes systematically worse performance than $\varepsilon = 10^{-5}$ across all five benchmarks we tested."*

A single hyper-parameter buried in the optimiser, six orders of magnitude smaller than the learning rate, makes a measurable empirical difference. Such is deep RL.

5. Ouyang, Wu, Jiang, et al. (2022) — Training language models to follow instructions with human feedback (InstructGPT)

arXiv:2203.02155 (March 2022).

Summary. The InstructGPT paper. PPO is used to fine-tune GPT-3 against a learned reward model trained on human preferences. The recipe (PPO + KL-to-reference penalty + per-token reward) is what underpins ChatGPT and every public LLM RLHF stack since.

Why it matters for this week. This is the production application of PPO that dominates the modern landscape. Read sections 3.1–3.3; the PPO loss they write is exactly our `ppo_loss`, with one extra reward-shaping term: $r_t \leftarrow r_t - \beta \log \pi_\theta(a_t \mid s_t)/\pi_{\text{ref}}(a_t \mid s_t)$.

Annotated key passage.

> *"We adapt the standard PPO algorithm to use a KL penalty between the policy and the SFT model as an additional reward shaping term, ensuring that the model does not drift too far from a well-behaved initialization."*

The KL term is what saves RLHF from the reward-hacking pathology. It is the exact mechanism we analysed in our safety angle, applied to a frontier-scale model. Week 18 implements this end-to-end.

Sourced from

`research/deeprl_core_sota.md` §2.6 and `research/llm_rl_sota.md` (RLHF section).